

Tutorial-5

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from tqdm import tqdm

# Step 1: Define the neural network architecture without dropout
class MLPWithMomentum(nn.Module):
    def __init__(self, input_size, hidden_size1, hidden_size2, output_size):
        super(MLPWithMomentum, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size1) # first linear layer
        self.relu1 = nn.ReLU() # relu
        self.fc2 = nn.Linear(hidden_size1, hidden_size2) # second linear layer
        self.relu2 = nn.ReLU() # relu
        self.fc3 = nn.Linear(hidden_size2, output_size) # last linear layer

    def forward(self, x):
        x=self.fc1(x)
        x = self.relu1(x)
        x=self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        return x

# Step 2: Load the CIFAR-10 dataset
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

train_dataset = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)

# Step 3: Create DataLoader for training and testing
batch_size = 64

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

# Step 4: Instantiate the model, loss function, and optimizer
input_size = 32 * 32 * 3 # input size for CIFAR-10
hidden_size1 = 256
hidden_size2 = 128
output_size = 10

model = MLPWithMomentum(input_size, hidden_size1, hidden_size2, output_size)
criterion = nn.CrossEntropyLoss()

momentum = [0.1, 0.9]
# Step 5: Training the model
num_epochs = 10

for mt in momentum:
    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=mt) # Use SGD with momentum
    print("----- MOMENTUM IS {} -----".format(mt))

    for epoch in range(num_epochs):
        model.train() # training mode here
        running_loss = 0.0
        total_train = 0
        correct_train = 0

        for inputs, labels in tqdm(train_loader, desc=f'Epoch {epoch + 1}/{num_epochs}'):
            inputs = inputs.view(inputs.size(0), -1) # Flatten the input images
            optimizer.zero_grad() # zero the gradient
            outputs = model(inputs)
            loss = criterion(outputs, labels) # calculate the loss
            loss.backward()
            optimizer.step()

            running_loss += loss.item()

```

```

_, predicted = torch.max(outputs.data, 1)
total_train += labels.size(0)
correct_train += (predicted == labels).sum().item()

train_accuracy = correct_train / total_train
print(f'Training Loss: {running_loss / len(train_loader)}, Training Accuracy: {train_accuracy * 100:.2f}%')

# Step 6: Testing the model
model.eval() # Eval mode here
correct = 0
total = 0

with torch.no_grad():
    for inputs, labels in tqdm(test_loader, desc='Testing'):
        inputs = inputs.view(inputs.size(0), -1)
        outputs = model(inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = correct / total
print(f'Test Accuracy: {accuracy * 100:.2f}%')

```

📄 Downloading <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz> to ./data/cifar-10-python.tar.gz

100%|██████████| 170498071/170498071 [00:02<00:00, 58798522.37it/s]

Extracting ./data/cifar-10-python.tar.gz to ./data

Files already downloaded and verified

----- MOMENTUM IS 0.1 -----

```

Epoch 1/10: 100%|██████████| 782/782 [00:19<00:00, 39.84it/s]
Training Loss: 2.0301112101206082, Training Accuracy: 28.42%
Epoch 2/10: 100%|██████████| 782/782 [00:20<00:00, 37.62it/s]
Training Loss: 1.7357932631012118, Training Accuracy: 38.98%
Epoch 3/10: 100%|██████████| 782/782 [00:19<00:00, 40.53it/s]
Training Loss: 1.6222720385512428, Training Accuracy: 42.99%
Epoch 4/10: 100%|██████████| 782/782 [00:20<00:00, 38.39it/s]
Training Loss: 1.5473935678791817, Training Accuracy: 45.81%
Epoch 5/10: 100%|██████████| 782/782 [00:20<00:00, 38.53it/s]
Training Loss: 1.489552142217641, Training Accuracy: 47.73%
Epoch 6/10: 100%|██████████| 782/782 [00:20<00:00, 37.66it/s]
Training Loss: 1.4380499511728506, Training Accuracy: 49.86%
Epoch 7/10: 100%|██████████| 782/782 [00:19<00:00, 39.81it/s]
Training Loss: 1.391876481301949, Training Accuracy: 51.49%
Epoch 8/10: 100%|██████████| 782/782 [00:20<00:00, 37.27it/s]
Training Loss: 1.3504995401100734, Training Accuracy: 52.95%
Epoch 9/10: 100%|██████████| 782/782 [00:19<00:00, 40.76it/s]
Training Loss: 1.3122947351706913, Training Accuracy: 54.38%
Epoch 10/10: 100%|██████████| 782/782 [00:20<00:00, 38.47it/s]
Training Loss: 1.2775172818347316, Training Accuracy: 55.51%
Testing: 100%|██████████| 157/157 [00:03<00:00, 52.18it/s]
Test Accuracy: 51.31%

```

----- MOMENTUM IS 0.9 -----

```

Epoch 1/10: 100%|██████████| 782/782 [00:20<00:00, 38.27it/s]
Training Loss: 1.426473331268486, Training Accuracy: 49.67%
Epoch 2/10: 100%|██████████| 782/782 [00:19<00:00, 39.80it/s]
Training Loss: 1.2991669651340036, Training Accuracy: 54.48%
Epoch 3/10: 100%|██████████| 782/782 [00:20<00:00, 38.07it/s]
Training Loss: 1.2203434342923372, Training Accuracy: 57.04%
Epoch 4/10: 100%|██████████| 782/782 [00:20<00:00, 38.78it/s]
Training Loss: 1.1554301188272589, Training Accuracy: 59.12%
Epoch 5/10: 100%|██████████| 782/782 [00:20<00:00, 37.43it/s]
Training Loss: 1.0975711126918988, Training Accuracy: 61.11%
Epoch 6/10: 100%|██████████| 782/782 [00:20<00:00, 38.69it/s]
Training Loss: 1.037179423948688, Training Accuracy: 63.17%
Epoch 7/10: 100%|██████████| 782/782 [00:21<00:00, 36.89it/s]
Training Loss: 0.9898054663025205, Training Accuracy: 64.83%
Epoch 8/10: 100%|██████████| 782/782 [00:20<00:00, 38.88it/s]
Training Loss: 0.939070142977073, Training Accuracy: 66.67%
Epoch 9/10: 100%|██████████| 782/782 [00:20<00:00, 38.16it/s]
Training Loss: 0.89597546596613, Training Accuracy: 68.02%
Epoch 10/10: 100%|██████████| 782/782 [00:19<00:00, 40.85it/s]
Training Loss: 0.847272159498366, Training Accuracy: 69.97%
Testing: 100%|██████████| 157/157 [00:04<00:00, 38.63it/s]Test Accuracy: 52.90%

```

